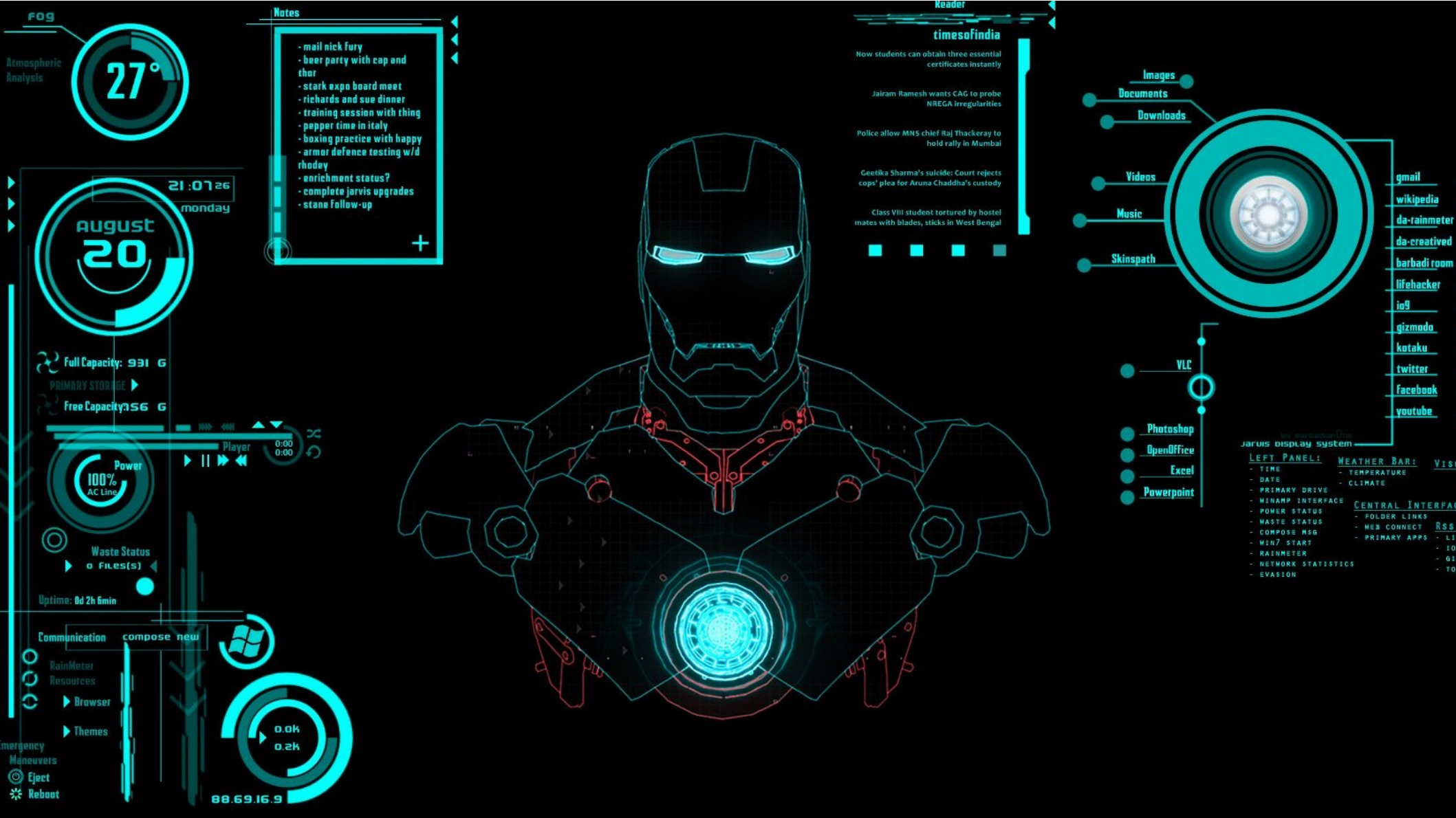




02.Desenhando em 2D – Parte 1

Prof. Alexandre Krohn

Roteiro

- Ponto / Pixel
- Retas
 - Anti-Aliasing

Roteiro

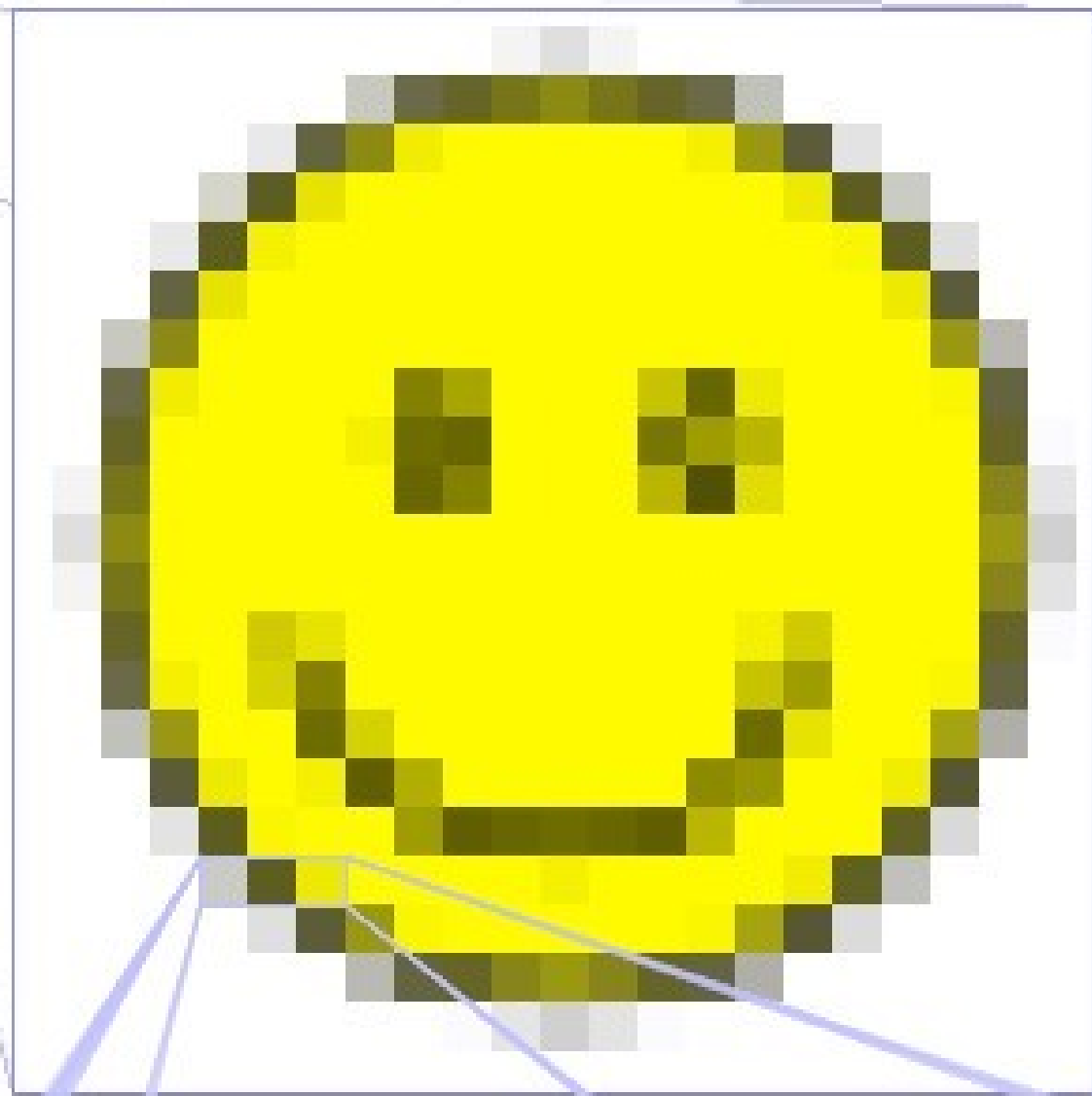
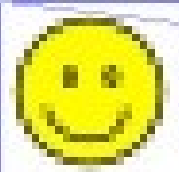
- Ponto / Pixel
- Retas
 - Anti-Aliasing

O ponto

- O ponto de partida de programas gráficos é a capacidade de desenhar pontos coloridos na tela
- Cada ponto é representado por um **pixel**.
- Pixel é uma palavra formada pela junção de **P**icture + **E**lement

Pixel

- O pixel é a **unidade mínima** de uma imagem digital
- Cada pixel é formado por 3 componentes de cor, **R**, **G** e **B**
- Os pixels podem também apresentar a propriedade de transparência (Alpha) : **RGBA**



RED 80%	RED 36%	RED 93%
GREEN 80%	GREEN 36%	GREEN 91%
BLUE 77%	BLUE 13%	BLUE 0%

Pixels por polegada?

- Pixels não possuem tamanho fixo determinado
- Quanto maior a quantidade de pixels, maior a resolução da imagem
- Isso implica que “fazer caber” mais pixels em uma tela, eles devem ser menores!
- Mais pixels = melhor qualidade, maior tamanho = maior consumo de memória e recursos de processador
- A medida de resolução adotada é a pontos por polegada

DPI e PPI

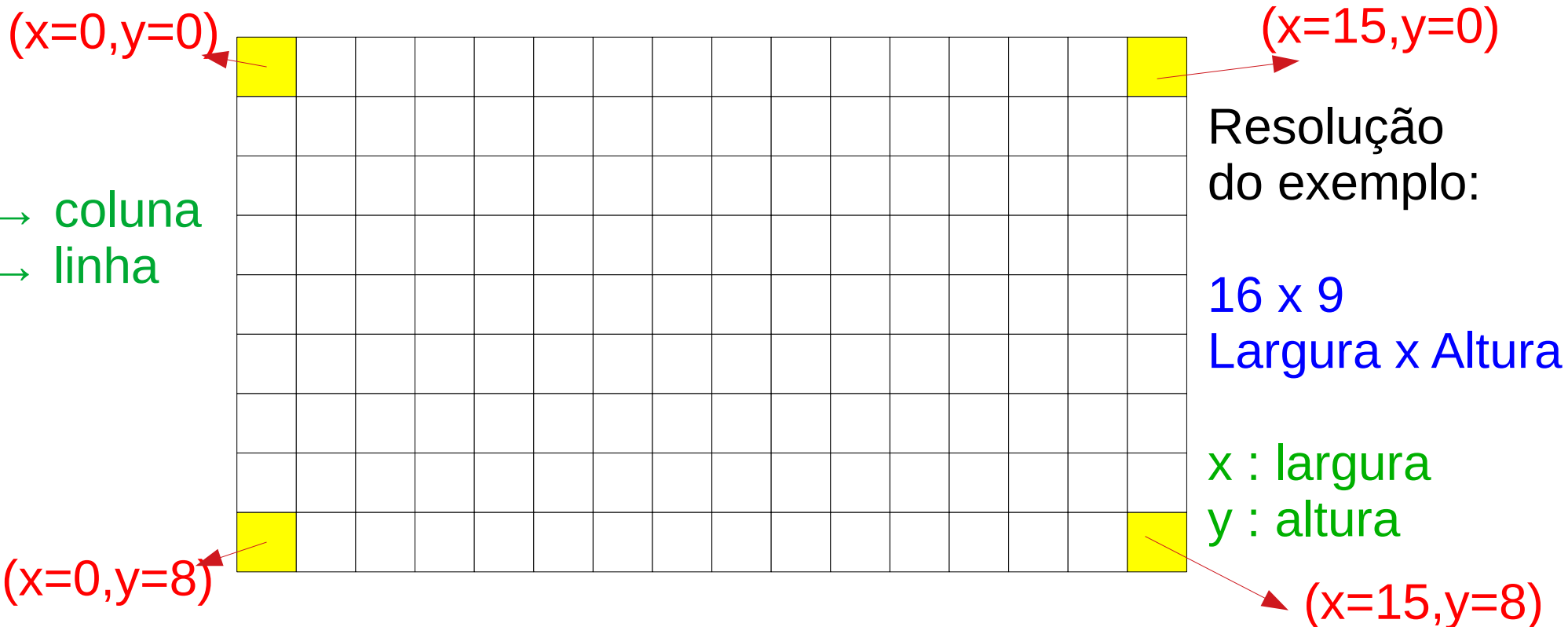
- **DPI** refere-se ao número de pontos impressos contido dentro de uma polegada de uma imagem impressa por uma **impressora**.
- **PPI** refere-se ao número de pixels contido dentro de uma polegada de uma imagem exibida no **monitor** do computador.

Resoluções de vídeo comuns

Nome	Largura x Altura	Formato	Total de pixels
QVGA	320 x 240	4:3	76.800
360p	640 x 360	16:9	230.400
VGA	640 x 480	4:3	307.200
VGA	720 x 480	4:3	345.600
SVGA	800 x 600	4:3	480.000
XGA	1024 x 768	4:3	786.432
WXGA-HD ¹ /720p	1280 x 720	16:9	921.600
WXGA ¹	1366 x 768	16:9	1.049.088
WXGA+	1440 x 900	16:10 (8:5)	1.296.000
UXGA	1600 x 900	16:9	1.440.000
UXGA++	1680 x 1050	16:10 (8:5)	1.764.000
Full HD/1080p	1920 x 1080	16:9	2.073.600
Full HD Ultra Wide ^[1]	2560 x 1080	21:9	2.764.800
WQHD	2560 x 1440	16:9	3.686.400
4K Ultra HD/2160p	3840 x 2160	16:9	8.294.400
8K UHD TV/4320p	7680 x 4320	16:9	33.177.600
10K UHD TV	10240x4320	21:9	44.236.800
10K UHD TV	10328x7760	4:3	80.145.280

Organização dos pixels na tela

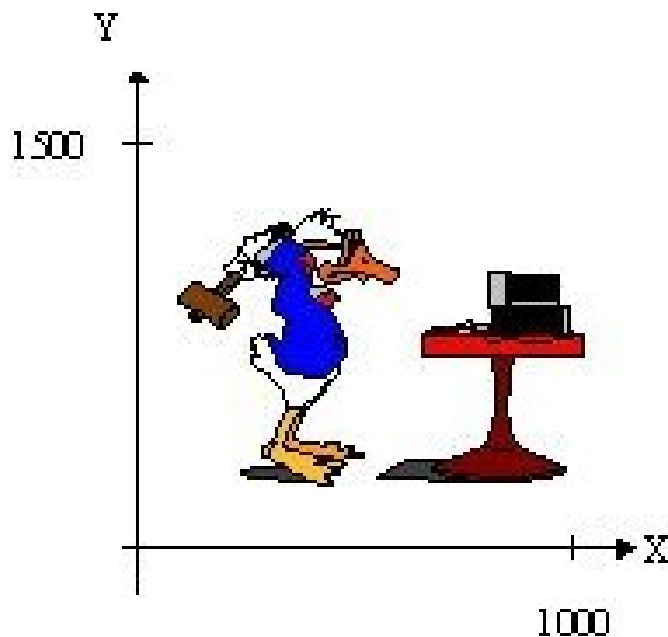
- Pixels são organizados como matrizes, começando na coordenada $(x=0, y=0)$ no canto superior esquerdo da tela, com x crescendo para baixo e y crescendo para a direita.



Ponto mínimo = $(0,0)$ Ponto máximo = $(altura - 1, largura - 1)$

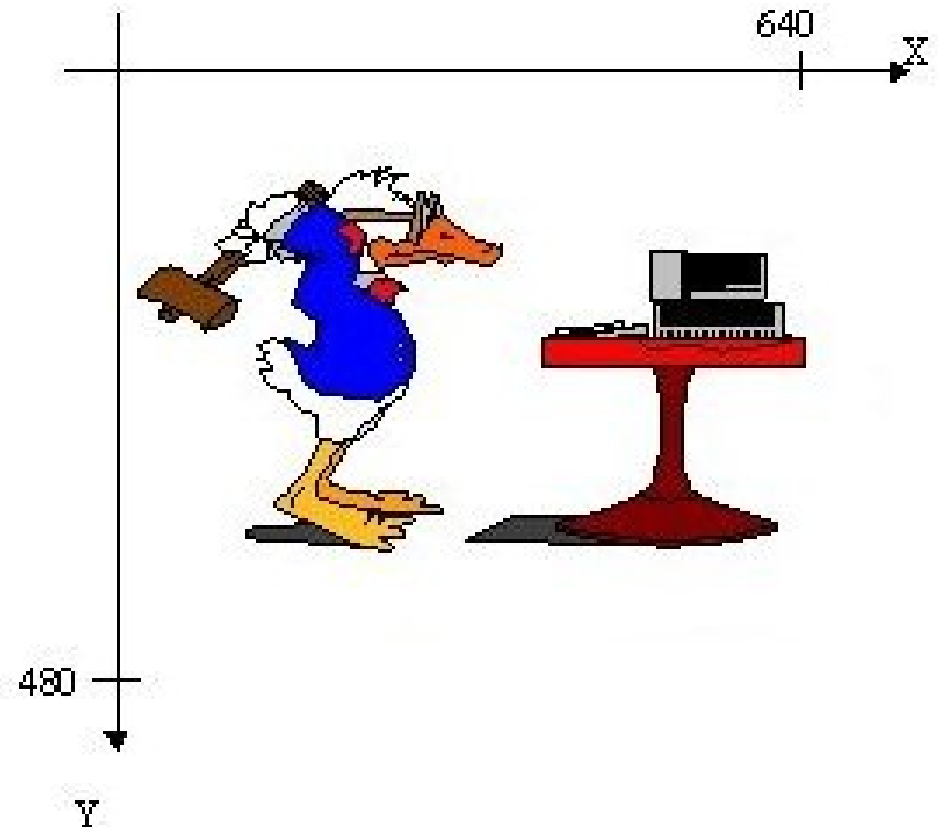
Disposição não-canônica

- Cuidado para não confundir com a disposição canônica, usada em álgebra, onde os valores de Y crescem de cima para baixo



Gráficos
Matemáticos

Monitores

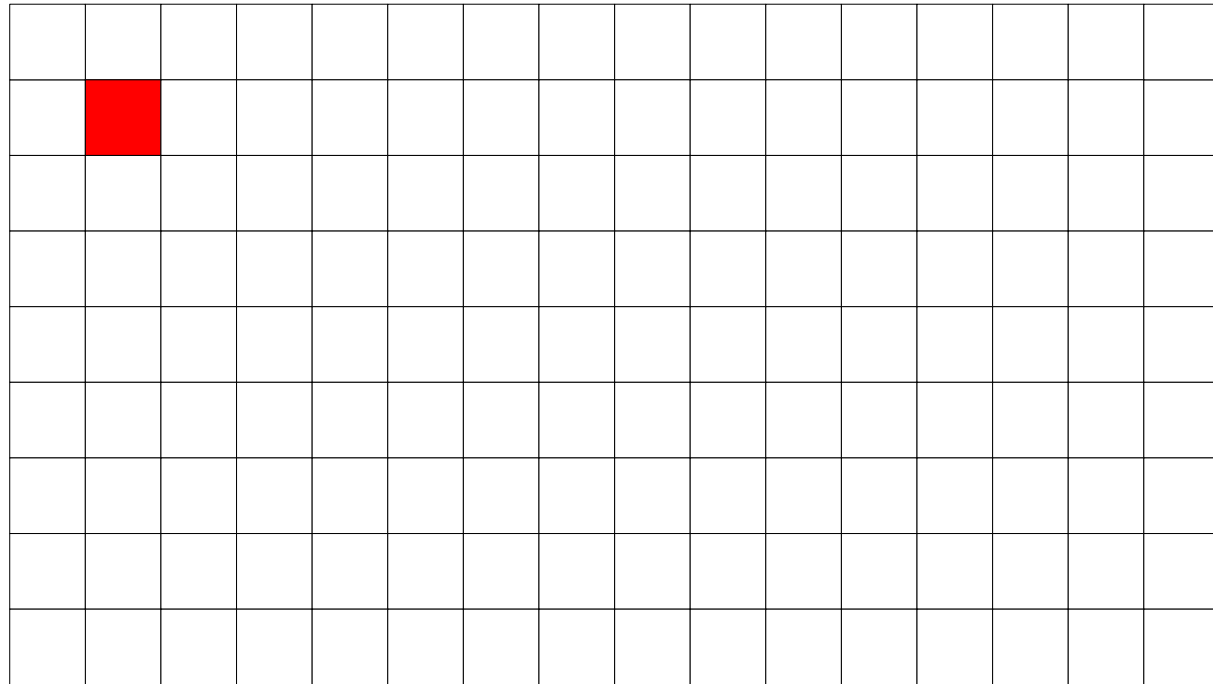


Desenhando pontos

- Partindo-se do princípio que existam duas **funções fictícias**:
- **RGB(r, g, b)** → retorna uma cor no formato solicitado
 - Valores para **r**, **g** e **b** variam de 0 até 255
- **set_pixel(x, y, cor)** → “liga” um pixel na tela com a cor solicitada
 - Valores para **x** variam de 0 até a **largura** da imagem - 1,
 - Valores para **y** variam de 0 até a **altura** da imagem - 1

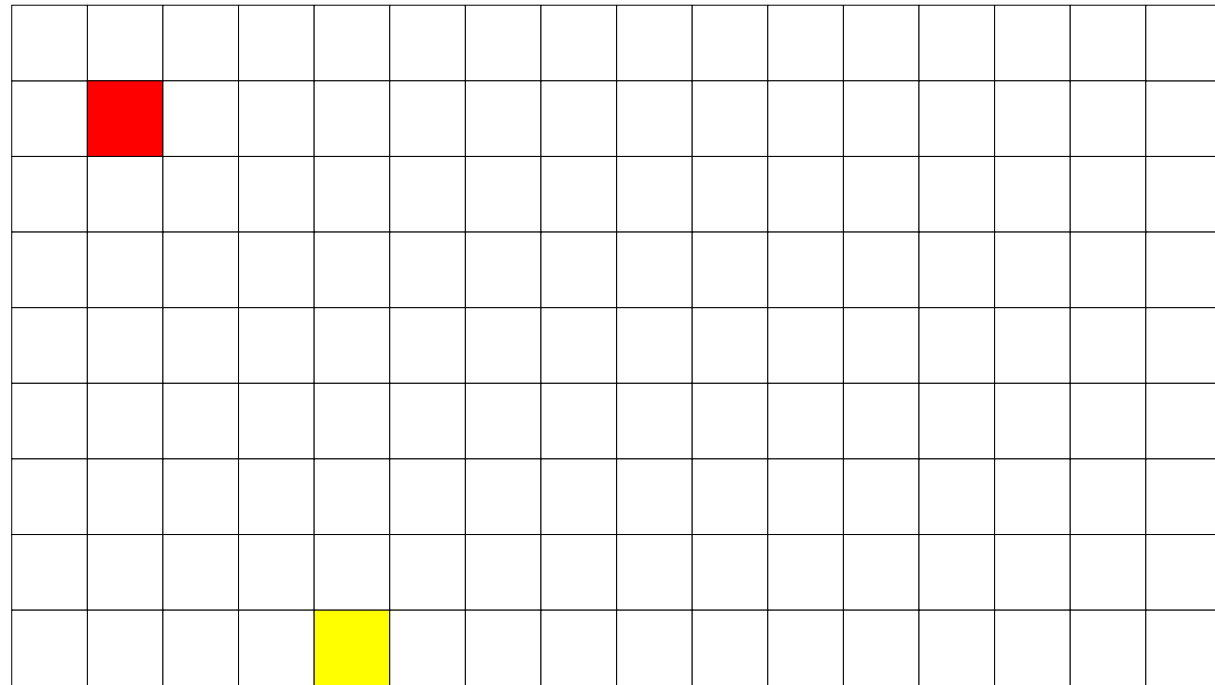
Desenhando pontos

`set_pixel(1,1,RGB(255,0,0))`



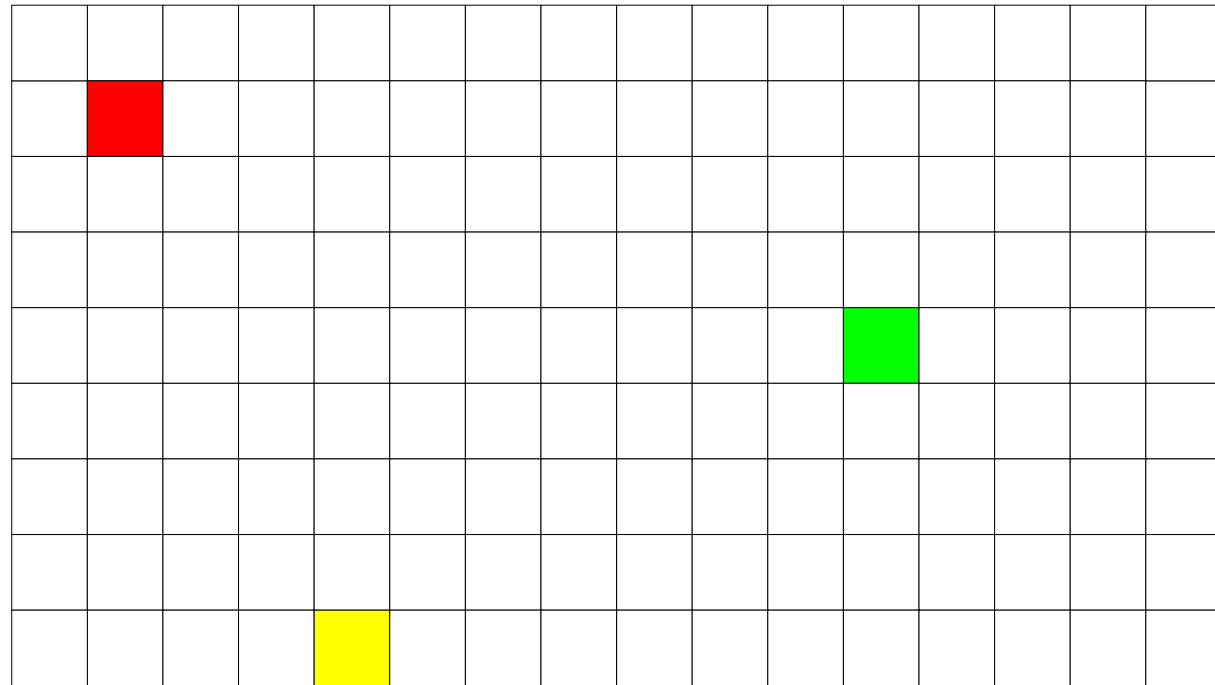
Desenhando pontos

```
set_pixel(1,1,RGB(255,0,0))  
set_pixel(8,4,RGB(255,255,0))
```



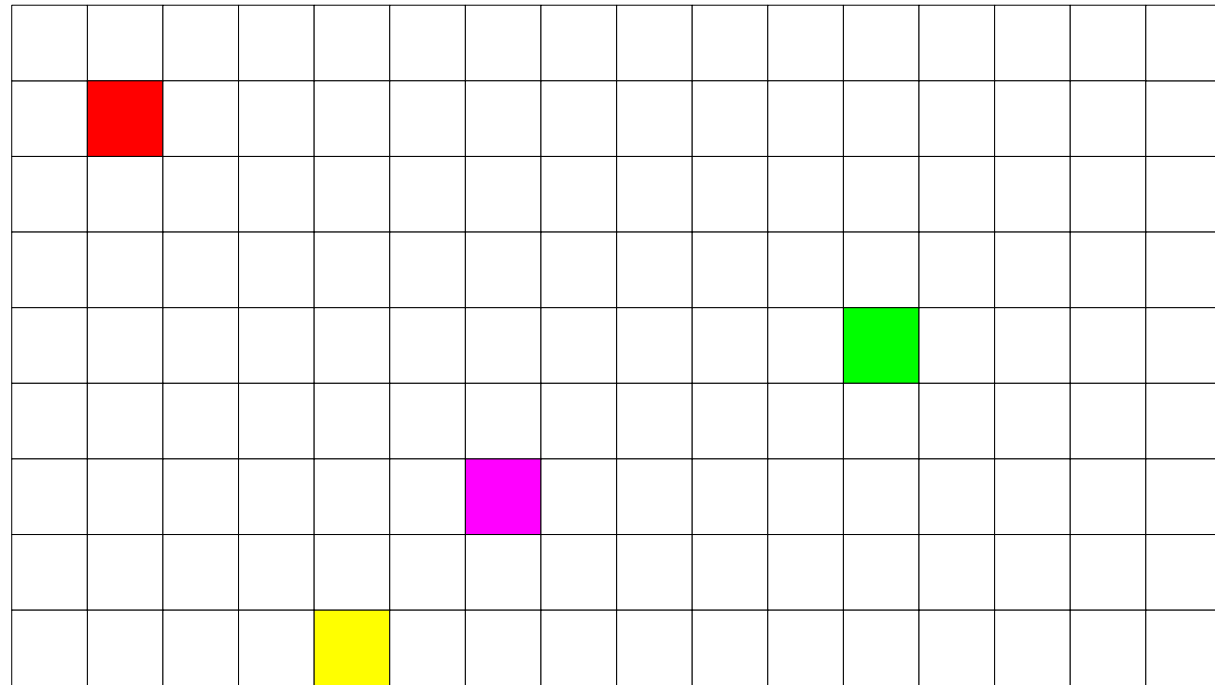
Desenhando pontos

```
set_pixel(1,1,RGB(255,0,0))  
set_pixel(8,4,RGB(255,255,0))  
set_pixel(4,11,RGB(0,255,0))
```



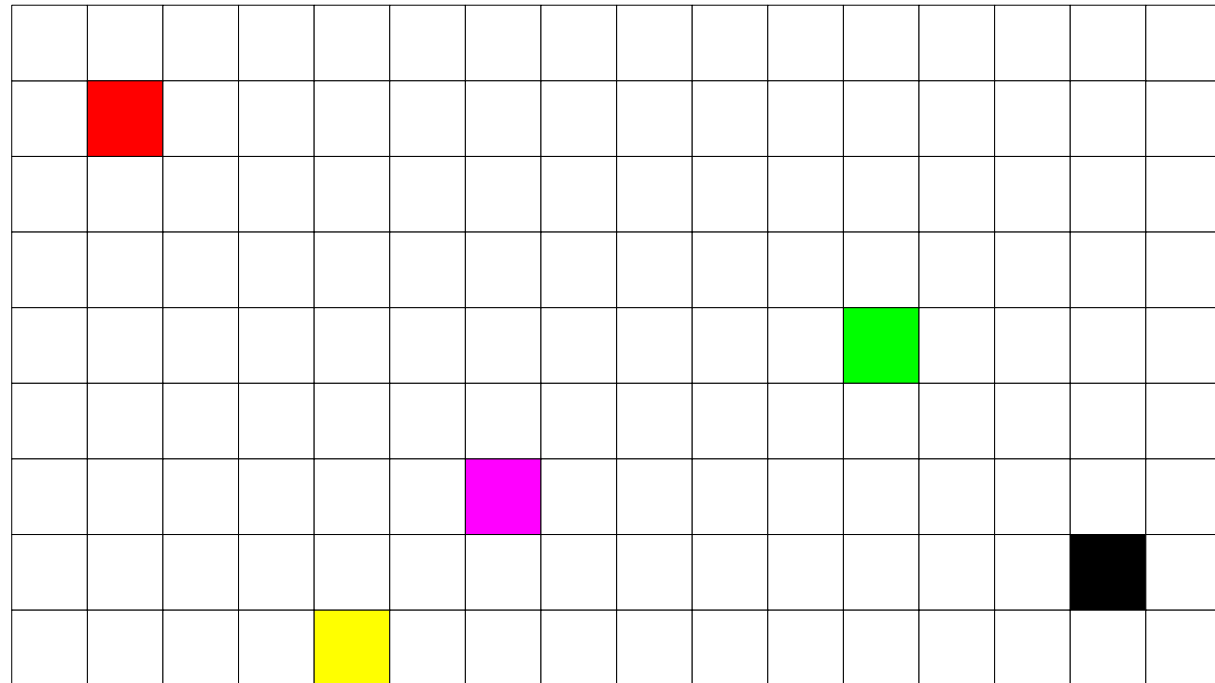
Downloaded from <https://www.cambridge.org/core>. University of Cambridge, on 02 Jun 2020 at 10:00:00, subject to the Cambridge Core terms of use, available at <https://www.cambridge.org/core/terms>. <https://doi.org/10.1017/9781009051625.007>

```
pixel(1,1,RGB(255,0,0))
pixel(8,4,RGB(255,255,0))
pixel(4,11,RGB(0,255,0))
pixel(6,6,RGB(255,0,255))
```



Desenhando pontos

```
set_pixel(1,1,RGB(255,0,0))  
set_pixel(8,4,RGB(255,255,0))  
set_pixel(4,11,RGB(0,255,0))  
set_pixel(6,6,RGB(255,0,255))  
set_pixel(7,14,RGB(0,0,0))
```



Roteiro

- Ponto / Pixel
- Retas
 - Anti-Aliasing

Retas

- A partir dos pontos, podemos desenhar as retas
- Retas horizontais e verticais são simples de desenhar: Basta usar um **laço de repetição**, do início ao fim da reta
- (E se a reta for de baixo para cima ou da direita para esquerda ?)

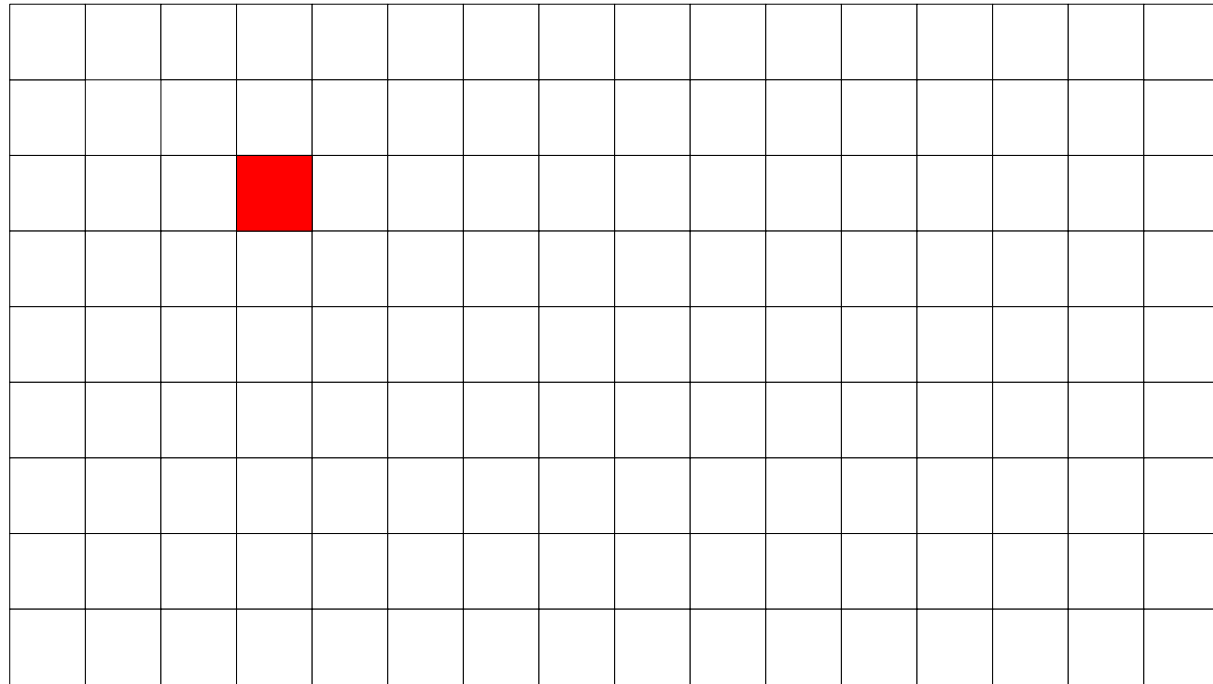
Reta Horizontal

- A partir dos pontos inicial e final da reta ($x=3, y=2$), ($x=8, y=2$), faz-se um laço variando apenas a coordenada y

para x de 3 até 8 faça

`set_pixel(x,2,RGB(255,0,0))`

fimpara



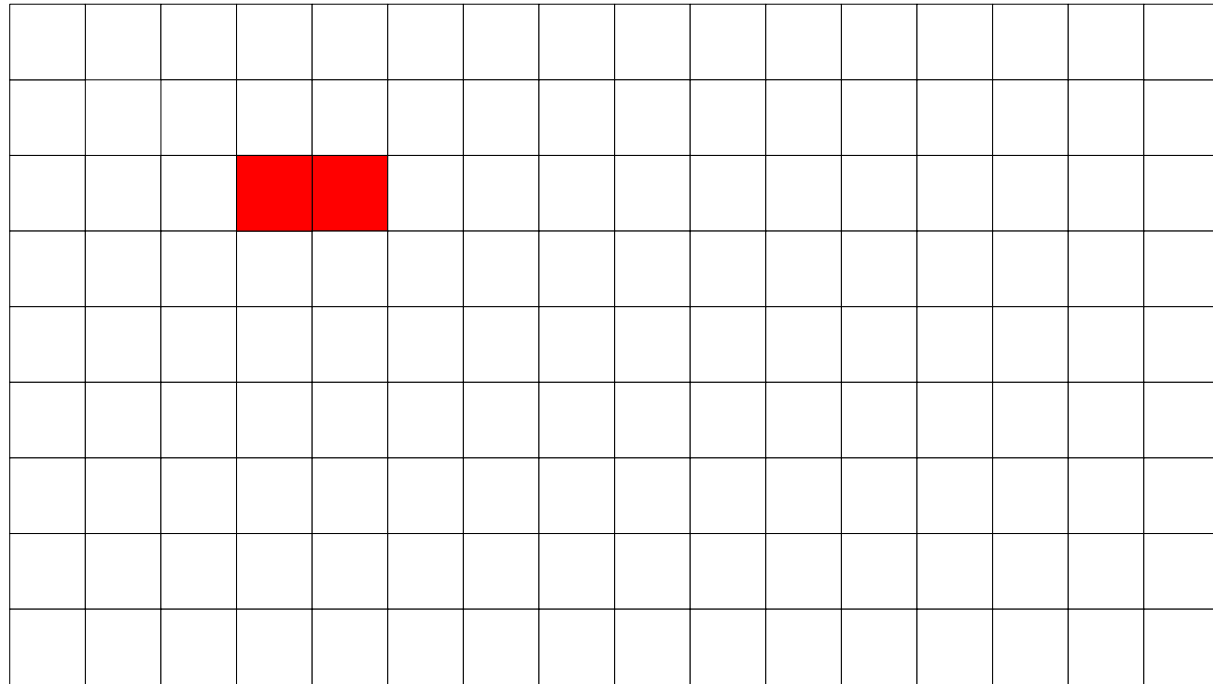
Reta Horizontal

- A partir dos pontos inicial e final da reta ($x=3, y=2$), ($x=8, y=2$), faz-se um laço variando apenas a coordenada y

para x de 3 até 8 faça

`set_pixel(x,2,RGB(255,0,0))`

fimpara



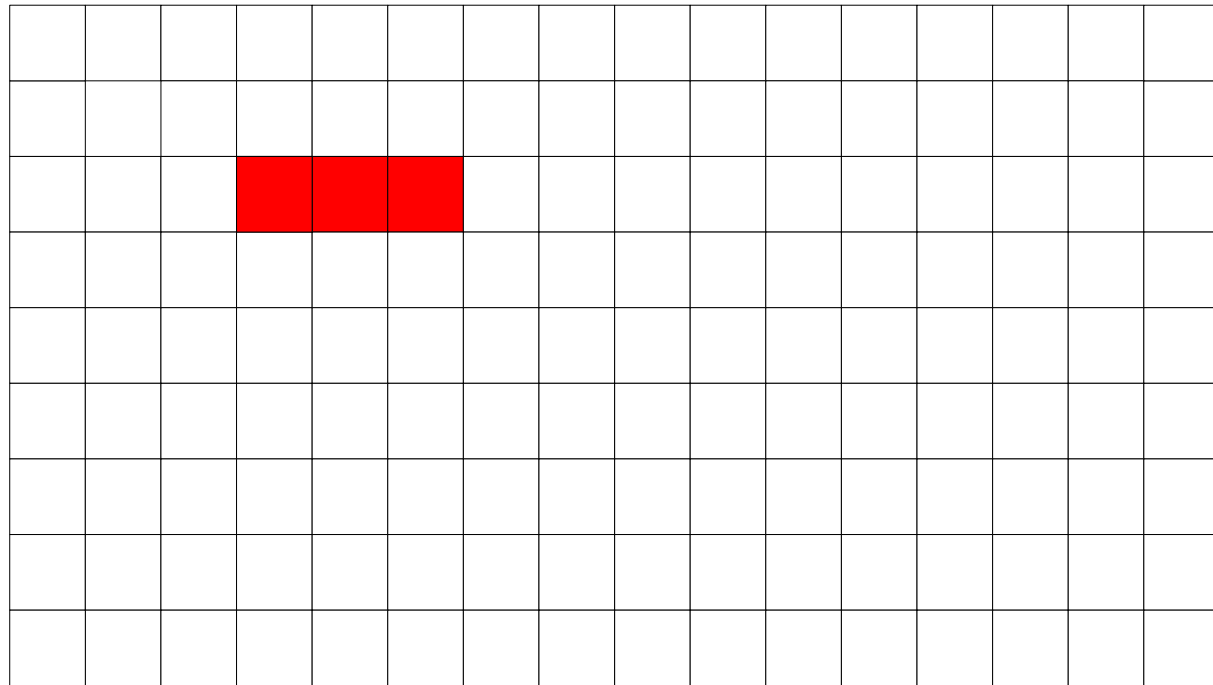
Reta Horizontal

- A partir dos pontos inicial e final da reta ($x=3, y=2$), ($x=8, y=2$), faz-se um laço variando apenas a coordenada y

para x de 3 até 8 faça

`set_pixel(x,2,RGB(255,0,0))`

fimpara



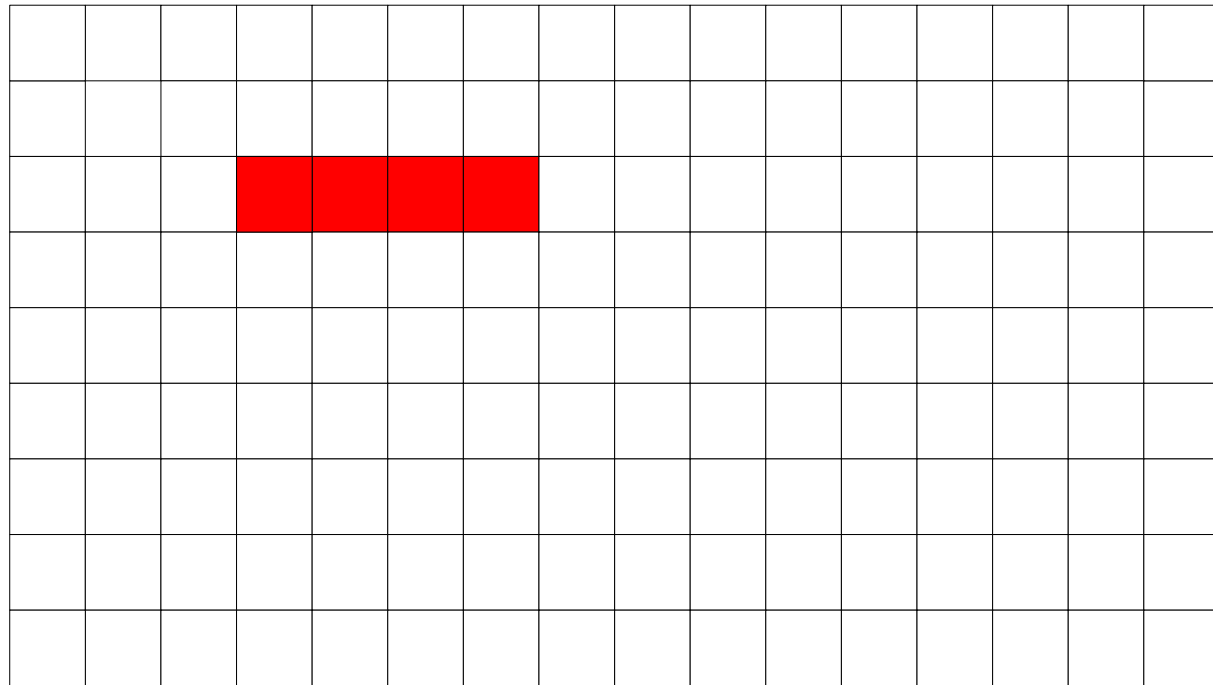
Reta Horizontal

- A partir dos pontos inicial e final da reta ($x=3, y=2$), ($x=8, y=2$), faz-se um laço variando apenas a coordenada y

para x de 3 até 8 faça

`set_pixel(x,2,RGB(255,0,0))`

fimpara



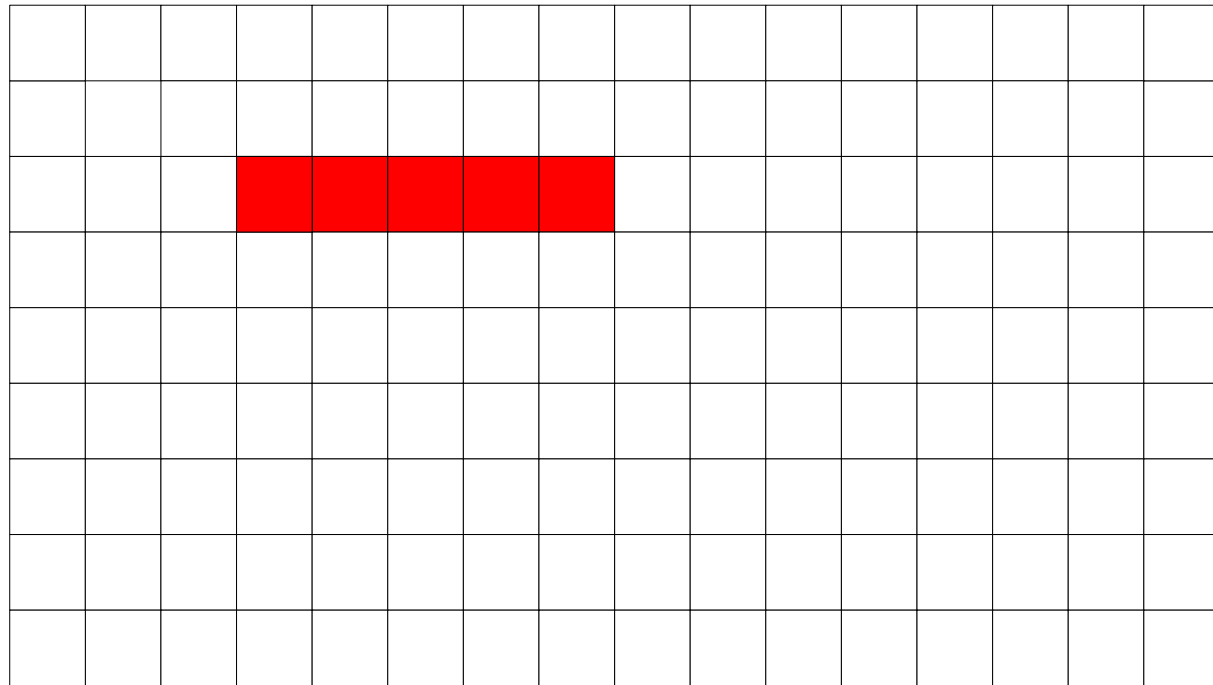
Reta Horizontal

- A partir dos pontos inicial e final da reta ($x=3, y=2$), ($x=8, y=2$), faz-se um laço variando apenas a coordenada y

para x de 3 até 8 faça

`set_pixel(x,2,RGB(255,0,0))`

fimpara



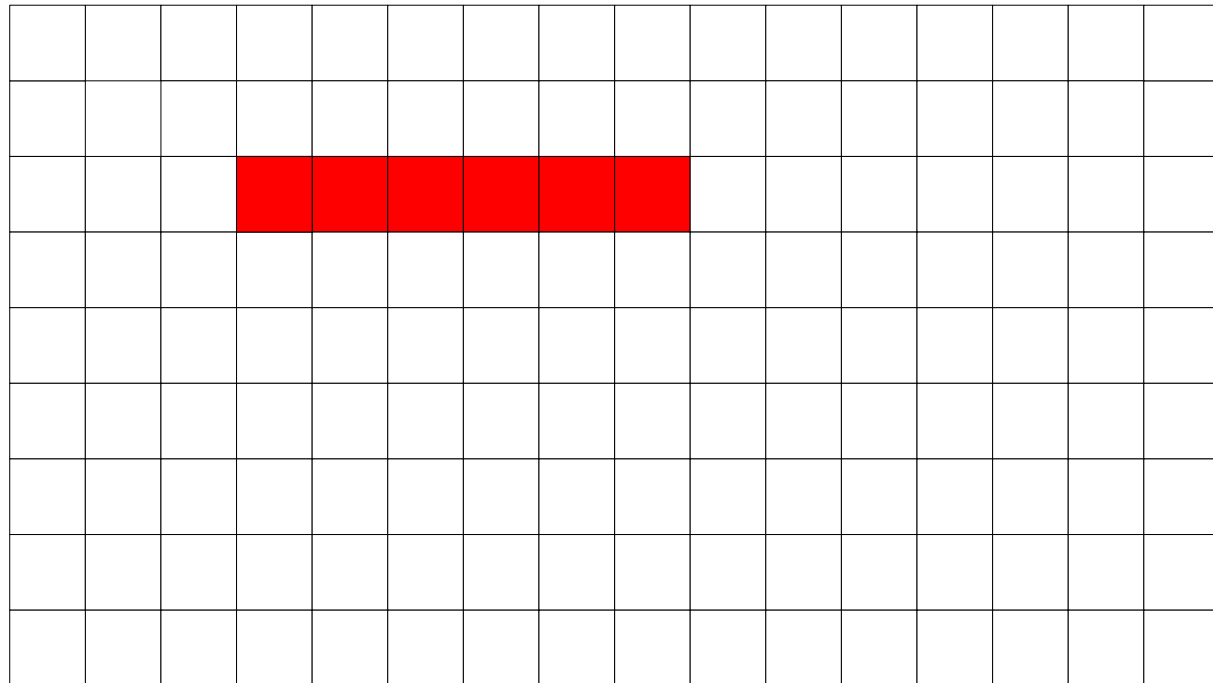
Reta Horizontal

- A partir dos pontos inicial e final da reta ($x=3, y=2$), ($x=8, y=2$), faz-se um laço variando apenas a coordenada y

para x de 3 até 8 faça

`set_pixel(x,2,RGB(255,0,0))`

fimpara



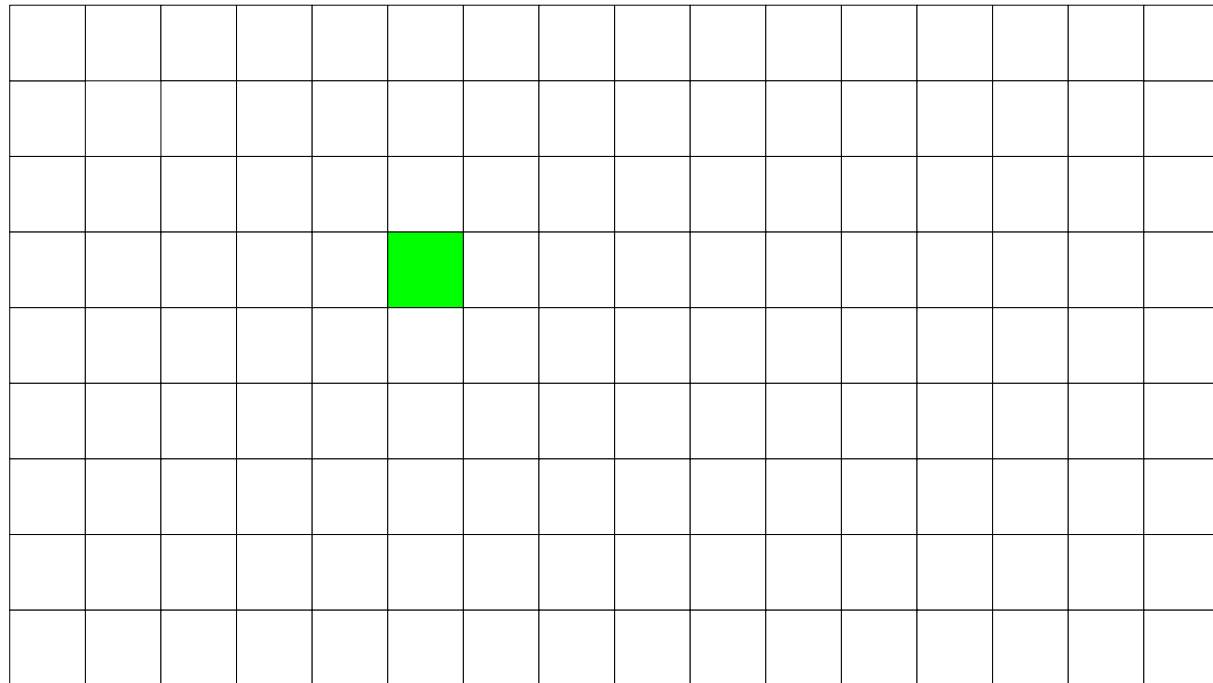
Reta Vertical

- A partir dos pontos inicial e final da reta ($x=5, y=3$), ($x=5, y=7$), faz-se um laço variando apenas a coordenada y

para y de 3 até 7 faça

`set_pixel(5,y,RGB(0,255,0))`

fimpara



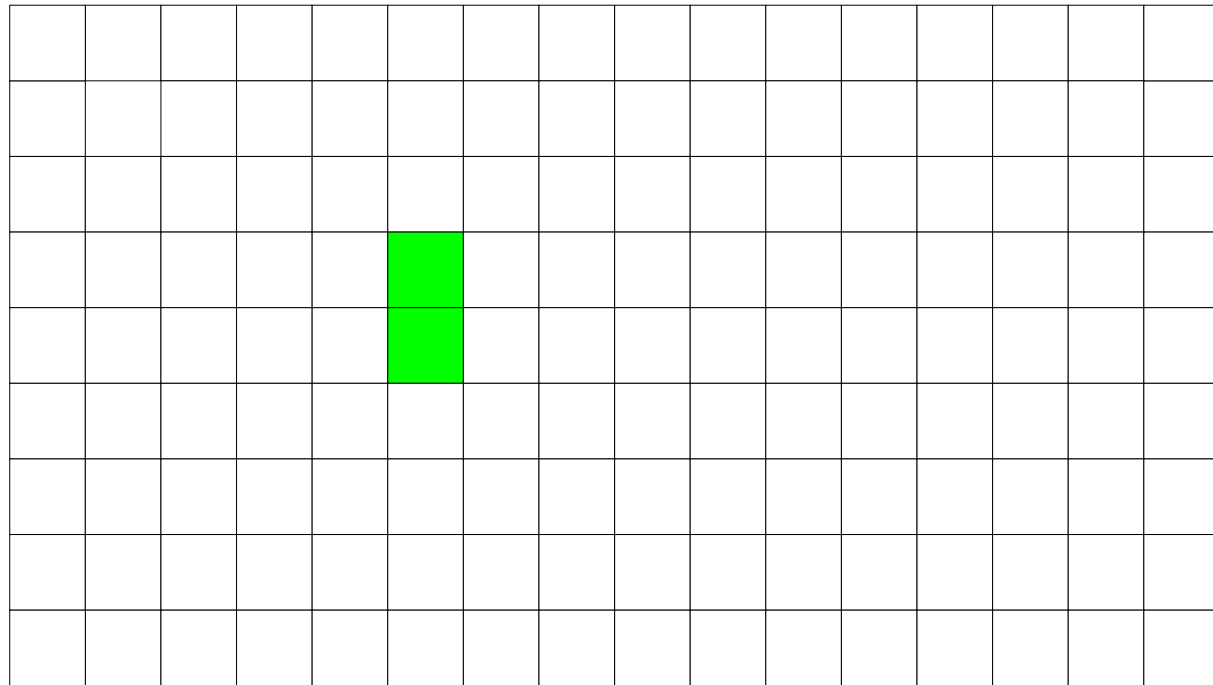
Reta Vertical

- A partir dos pontos inicial e final da reta ($x=5, y=3$), ($x=5, y=7$), faz-se um laço variando apenas a coordenada y

para y de 3 até 7 faça

`set_pixel(5,y,RGB(0,255,0))`

fimpara



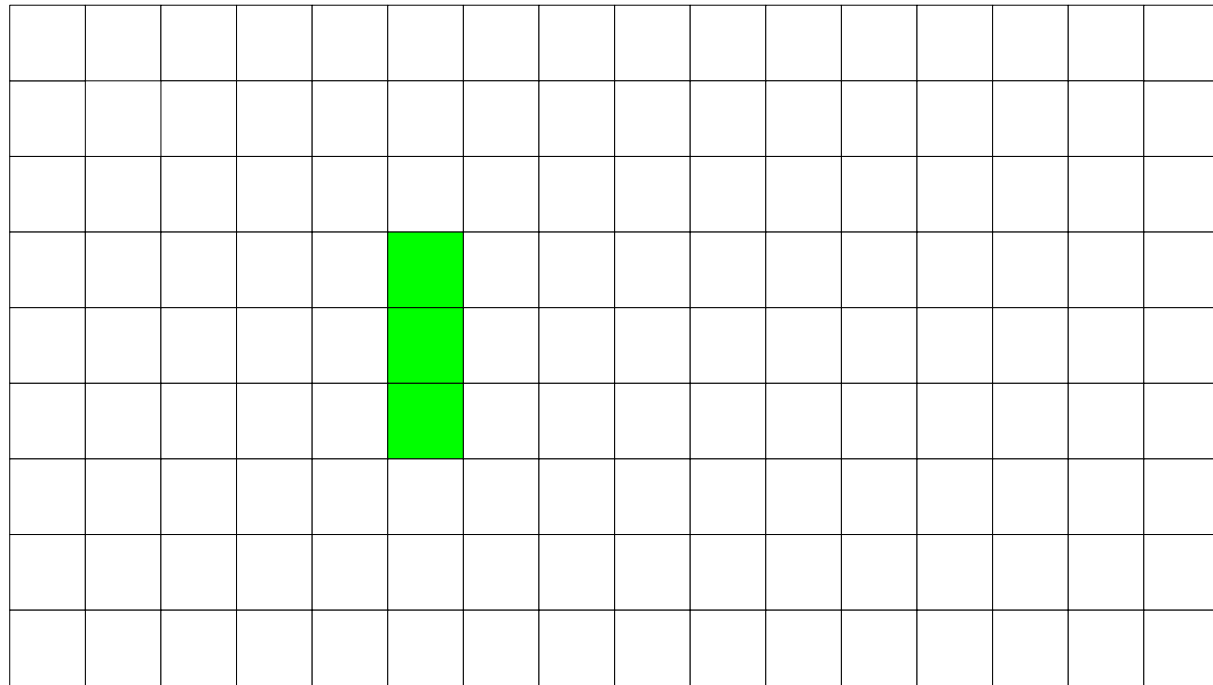
Reta Vertical

- A partir dos pontos inicial e final da reta ($x=5, y=3$), ($x=5, y=7$), faz-se um laço variando apenas a coordenada y

para y de 3 até 7 faça

`set_pixel(5,y,RGB(0,255,0))`

fimpara



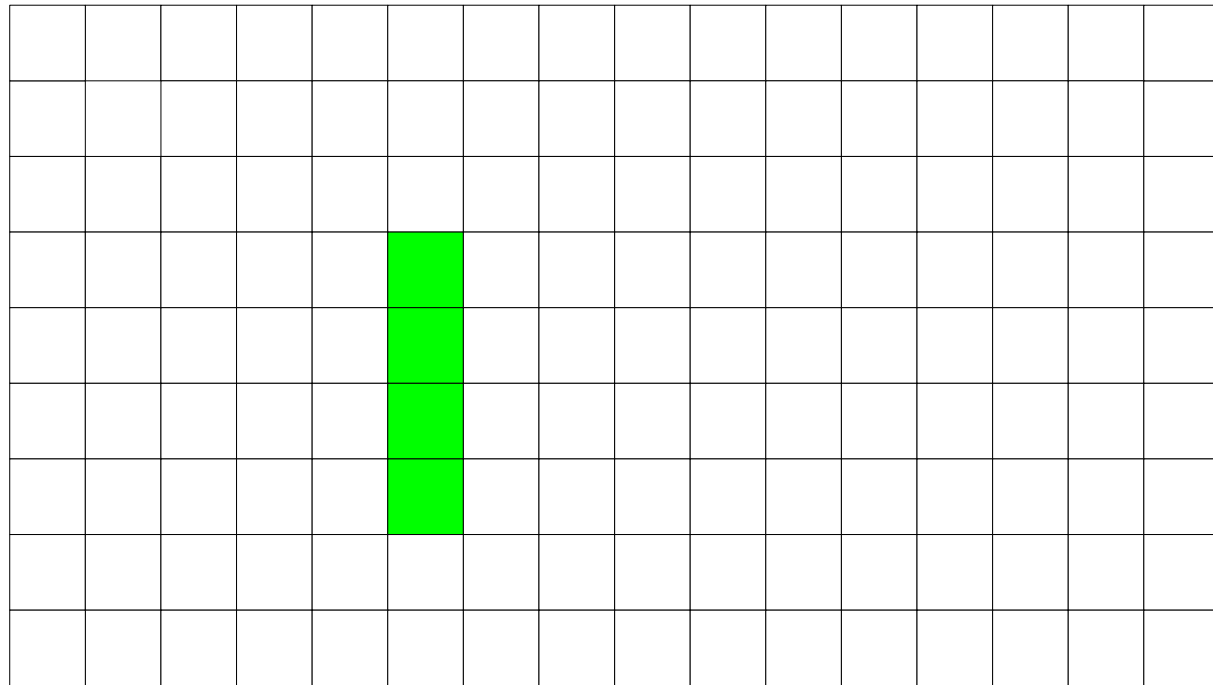
Reta Vertical

- A partir dos pontos inicial e final da reta ($x=5, y=3$), ($x=5, y=7$), faz-se um laço variando apenas a coordenada y

para y de 3 até 7 faça

`set_pixel(5,y,RGB(0,255,0))`

fimpara



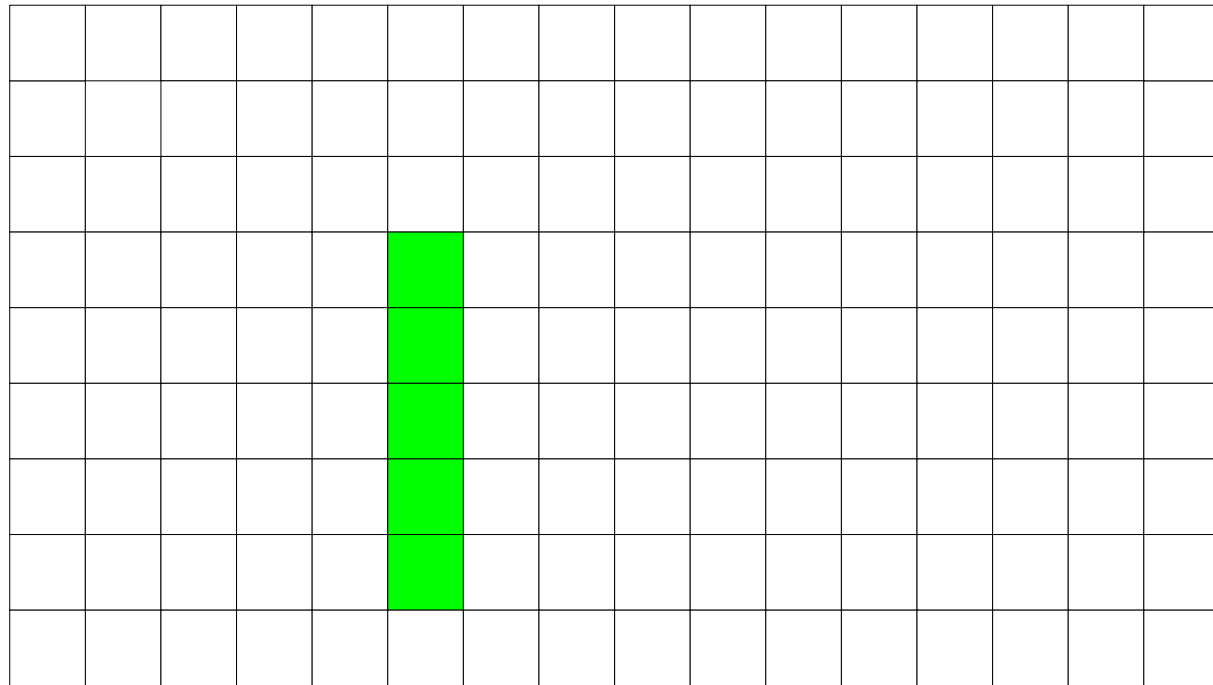
Reta Vertical

- A partir dos pontos inicial e final da reta ($x=5, y=3$), ($x=5, y=7$), faz-se um laço variando apenas a coordenada y

para y de 3 até 7 faça

`set_pixel(5,y,RGB(0,255,0))`

fimpara

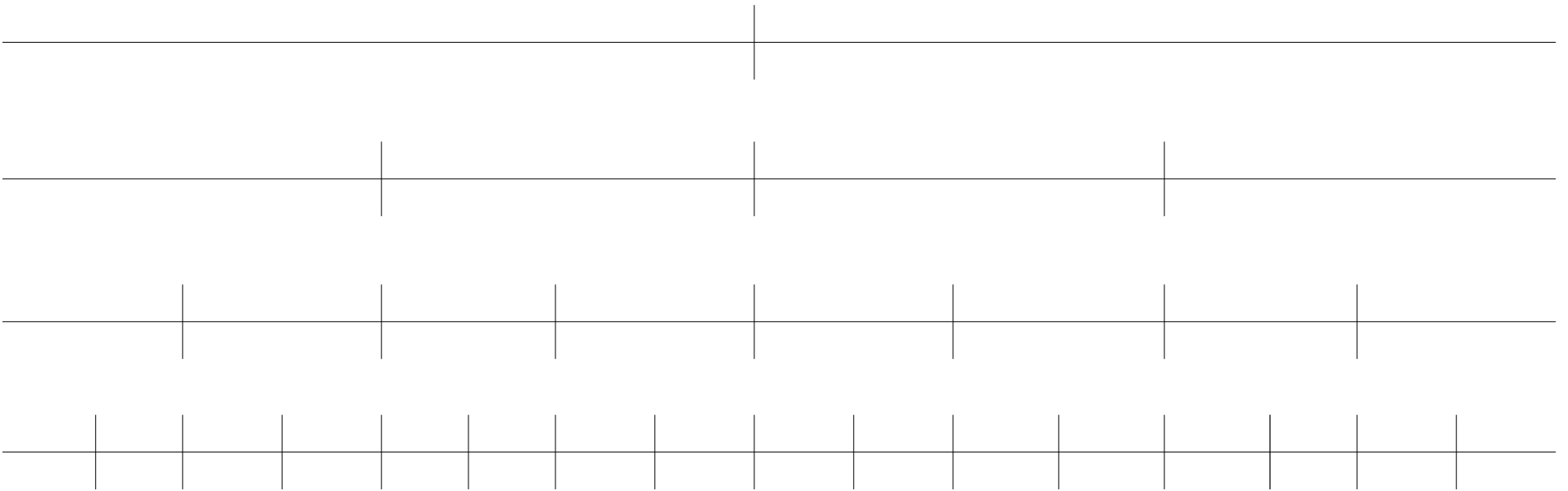


Retas Diagonais

- Retas diagonais são mais complicadas, por um motivo simples:
 - No mundo real, retas são **contínuas**:
-

Retas Diagonais

- Uma reta pode ser sempre dividida em segmentos menores:



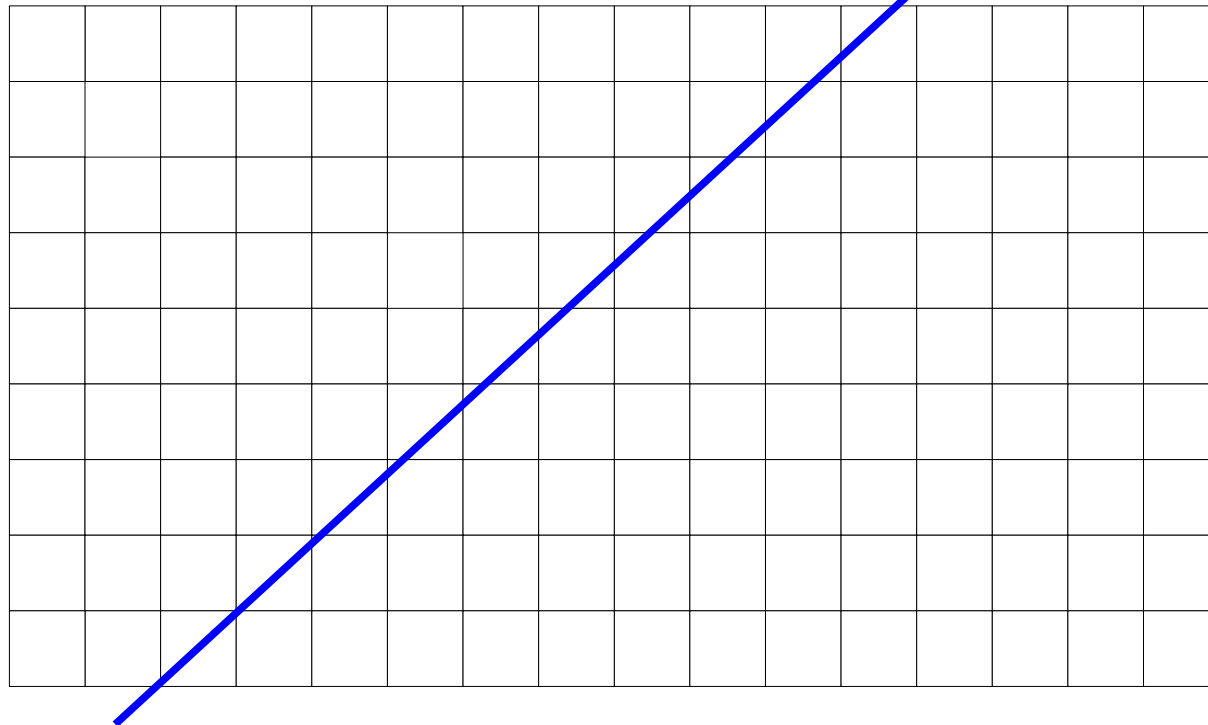
Isso pode ser repetido infinitas vezes

Retas diagonais

- Mas na tela do computador, as retas são representadas por pixels.
- Há um **número limitado de pixels** na tela, e portanto na reta desenhada: Retas desenhadas na tela são compostas de **valores discretos**
- Isso faz com que a reta seja uma **aproximação** da reta real

Retas Diagonais

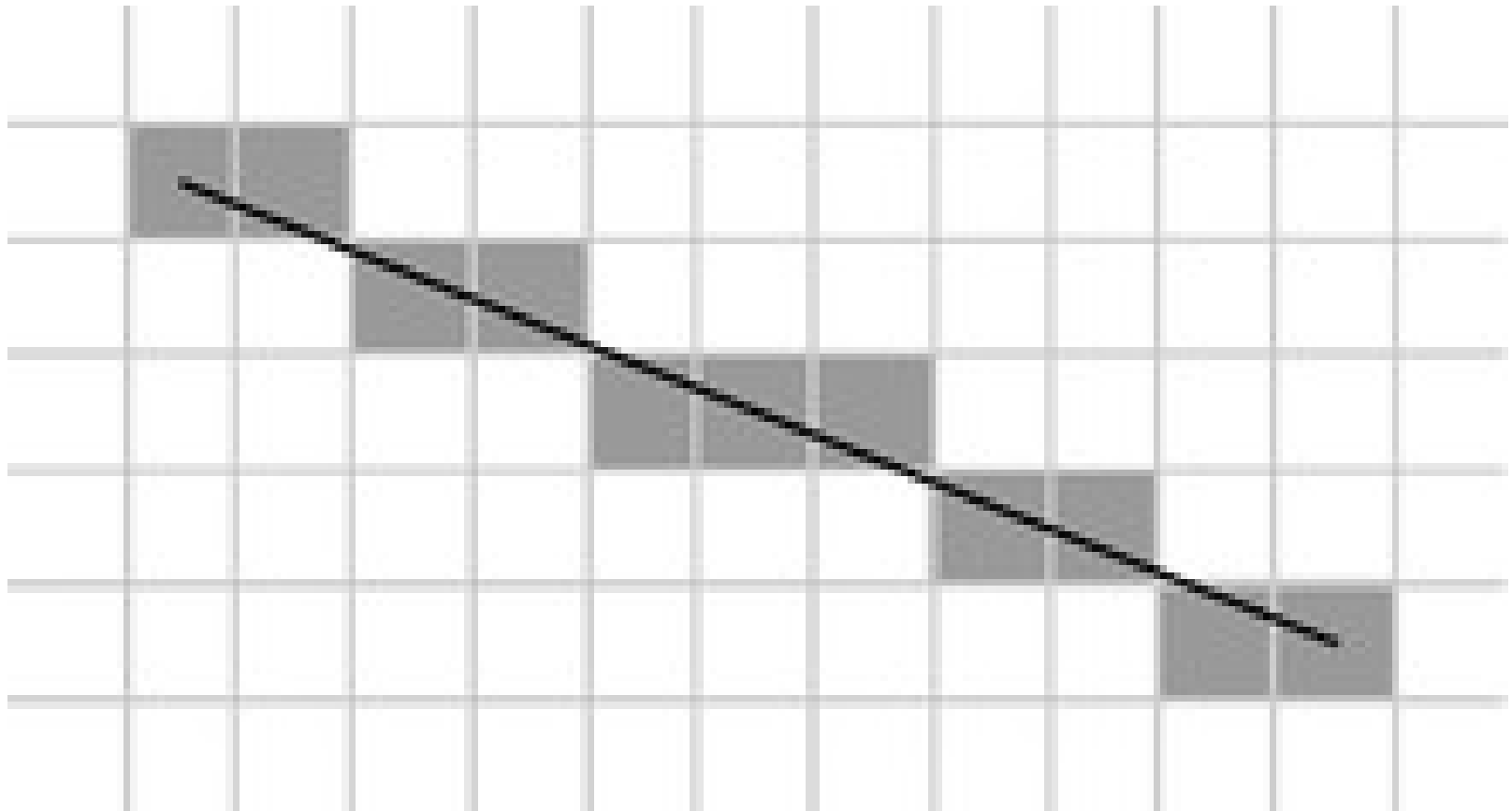
- Como determinar quais pixels devem ser ligados para representar essa reta?



$P1=(0,11)$
 $P2=(8,2)$

Retas diagonais

- O principal algoritmo utilizado para isso é o de Bresenham:



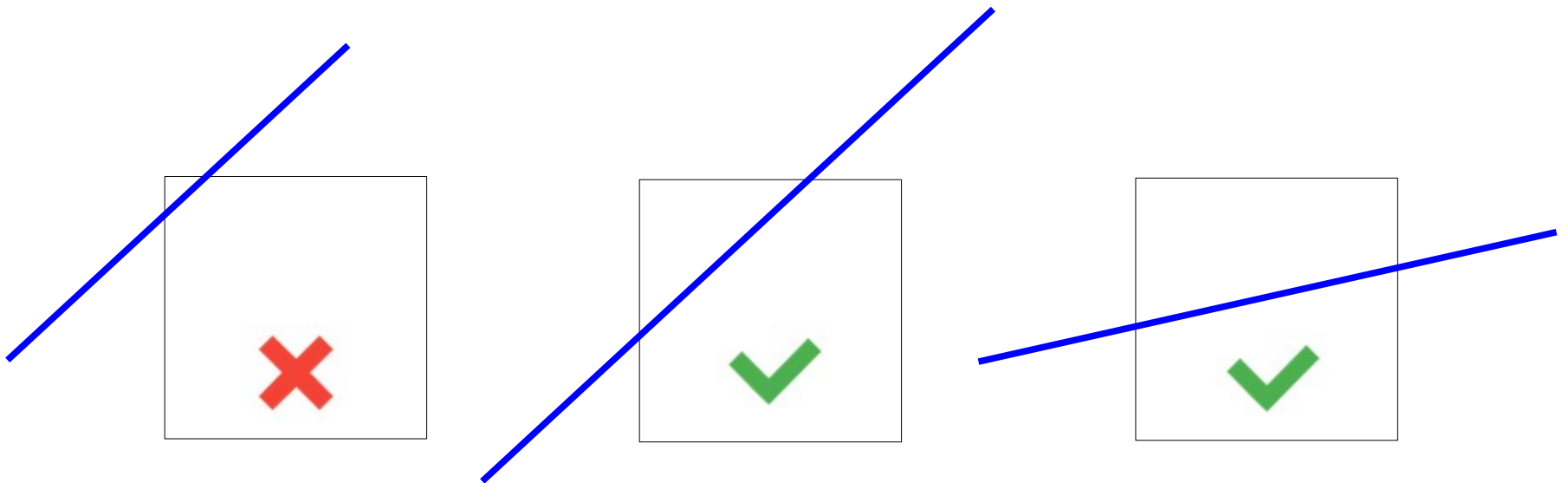
Algoritmo de Bresenham

- **Jack Elton Bresenham** nasceu em 1937, nos EUA. Trabalhou por 27 anos na IBM e depois como professor de Ciência da Computação na Winthrop University
- Em **1962**, criou o algoritmo de desenho de linhas retas para computadores



Algoritmo de Bresenham

- O desafio do algoritmo é calcular se um **pixel deve ou não ser ativado** na tela.
- Isso é feito analisando o **percentual de sua área** que é atravessado pela reta



Algoritmo de Bresenham

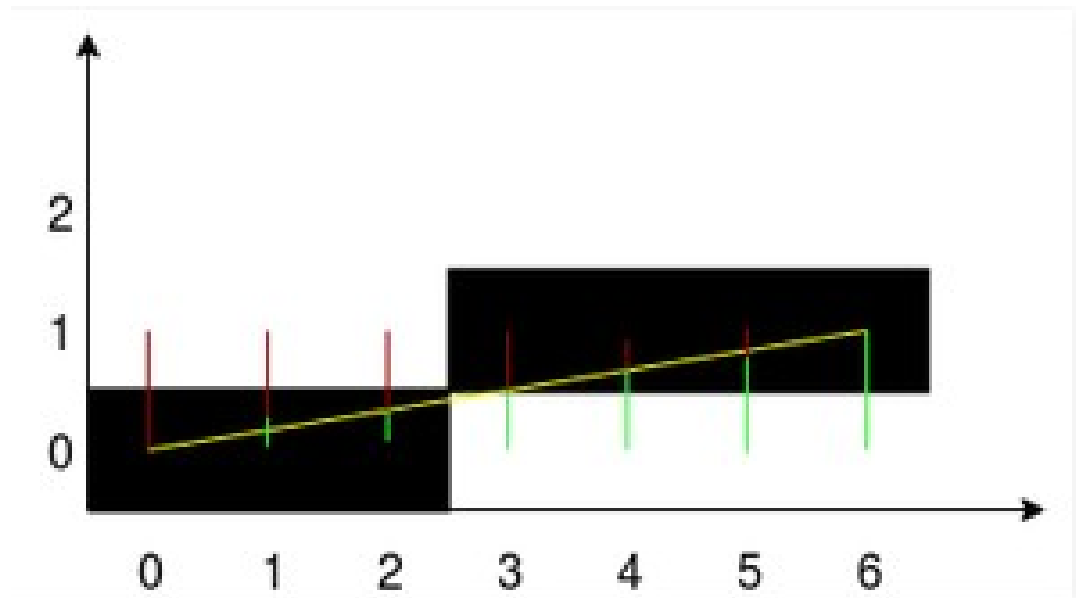
- Calcular **percentuais** envolvem cálculos com **ponto flutuante**.
- Cálculos com ponto flutuante são mais “caros” em computação : **Consomem mais recursos**
- Retas possuem centenas de pontos, as vezes milhares
- **Programas gráficos possuem muitas retas**

Algoritmo de Bresenham

- Jack **Bresenham** desenvolveu um algoritmo que usa **apenas adições, subtrações, e operações de deslocamento de bits (shift)** para calcular se um ponto pertence ou não a reta

Algoritmo de Bresenham

- Pegue um segmento e sua coordenada **inicial x**.
- Ao lado do X no loop é **adicionado 1** no final do segmento.
- **Em cada etapa, o erro é calculado : a distância entre a coordenada y real naquele local e a célula da grade mais próxima.**
- Se o erro **não ultrapassar** a metade da altura da célula, ela será preenchida.



O Algoritmo de Bresenham

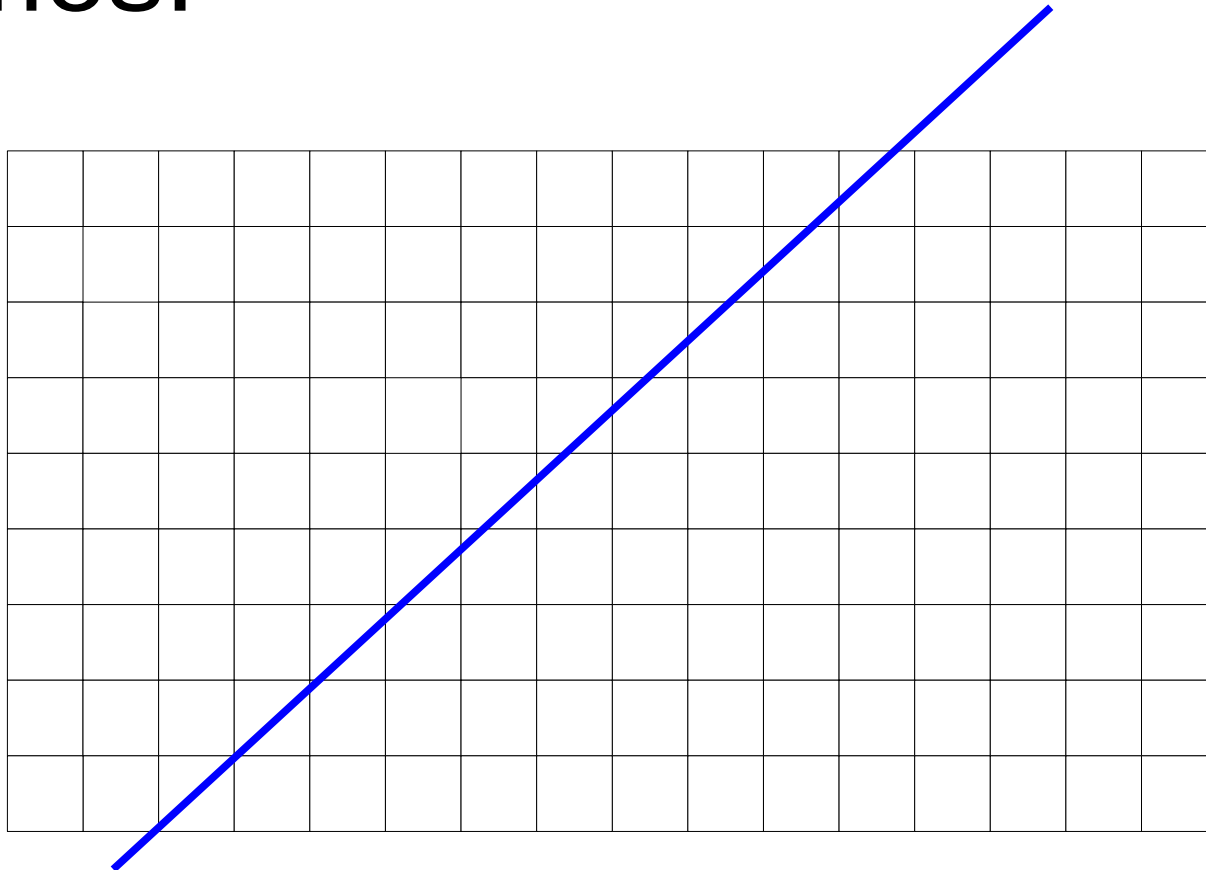
```
void bresenham(int x1, int y1, int x2, int y2){  
    int slope; // inclinação  
    int dx, dy, incE, incNE, d, x, y;  
    // Onde inverte a linha x1 > x2  
    if (x1 > x2){  
        bresenham(x2, y2, x1, y1);  
        return;  
    }  
    dx = x2 - x1;  
    dy = y2 - y1;  
  
    if (dy < 0){  
        slope = -1;  
        dy = -dy;  
    }  
    else{  
        slope = 1;  
    }  
}
```

Esse algoritmo funciona quando a reta é da esquerda para direita e de baixo para cima. Quando não for, inverte!

```
// Constante de Bresenham  
incE = 2 * dy; // shift-left  
incNE = 2 * dy - 2 * dx;  
d = 2 * dy - dx;  
y = y1;  
for (x = x1; x <= x2; x++){  
    putpixel(x, y);  
    if (d <= 0){  
        d += incE;  
    }  
    else{  
        d += incNE;  
        y += slope;  
    }  
}
```

Retas Diagonais

- Utilizando o algoritmo de Bresenham, obtemos:



$P1=(0,11)$

$P2=(8,2)$

$(8,2)$

$(7,3)$

$(6,4)$

$(5,5)$

$(4,6)$

$(4,7)$

$(3,8)$

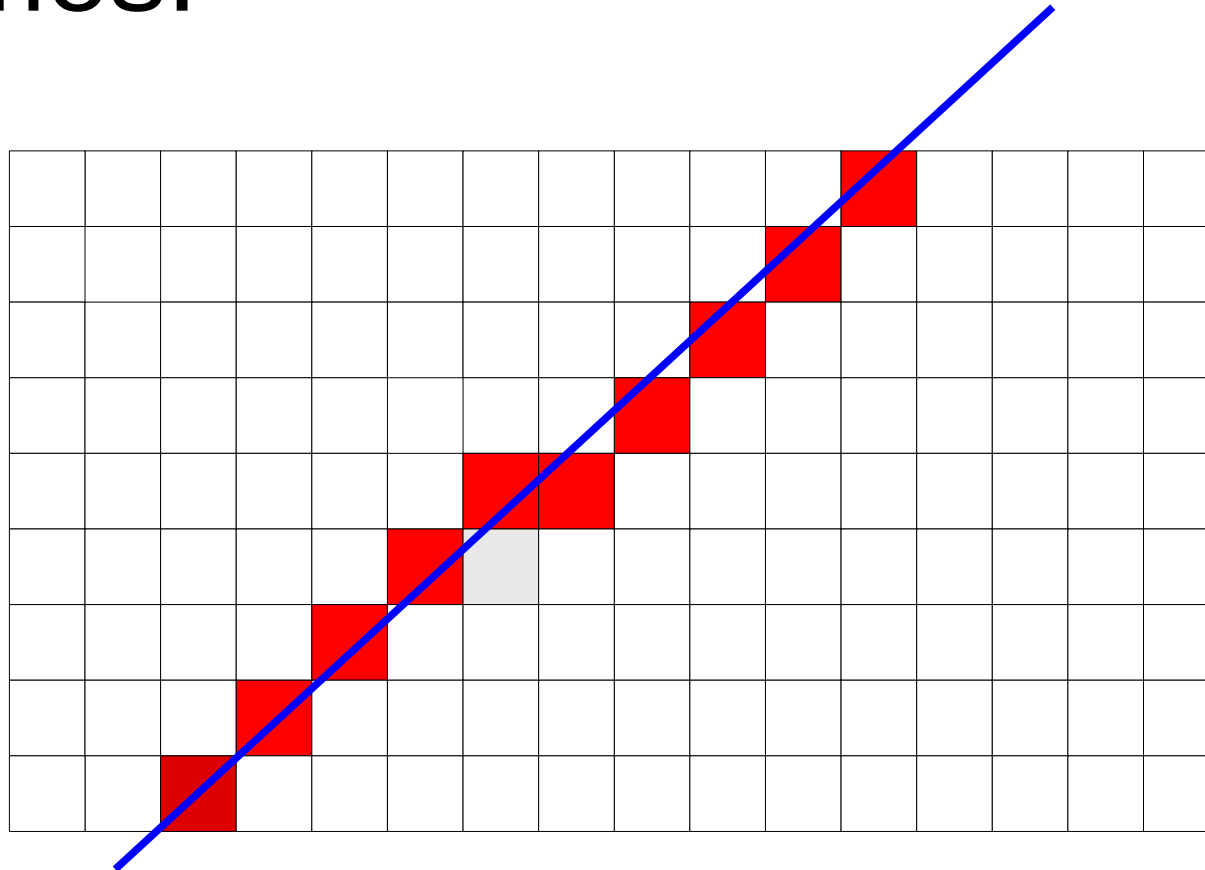
$(2,9)$

$(1,10)$

$(0,11)$

Retas Diagonais

- Utilizando o algoritmo de Bresenham, obtemos:



P1=(0,11)

P2=(8,2)

(8,2)

(7,3)

(6,4)

(5,5)

(4,6)

(4,7)

(3,8)

(2,9)

(1,10)

(0,11)

Algoritmo de Bresenham

- Atualmente, o algoritmo de Bresenham está implementado no **firmware das placas gráficas**

Algoritmo de Bresenham

- O algoritmo de **Bresenham** desenha linhas extremamente rápido, mas não executa *anti-aliasing*.
- Além disso, ele não pode lidar com nenhum caso em que os pontos finais da linha não se situem exatamente nos pontos inteiros da grade de pixels.

Roteiro

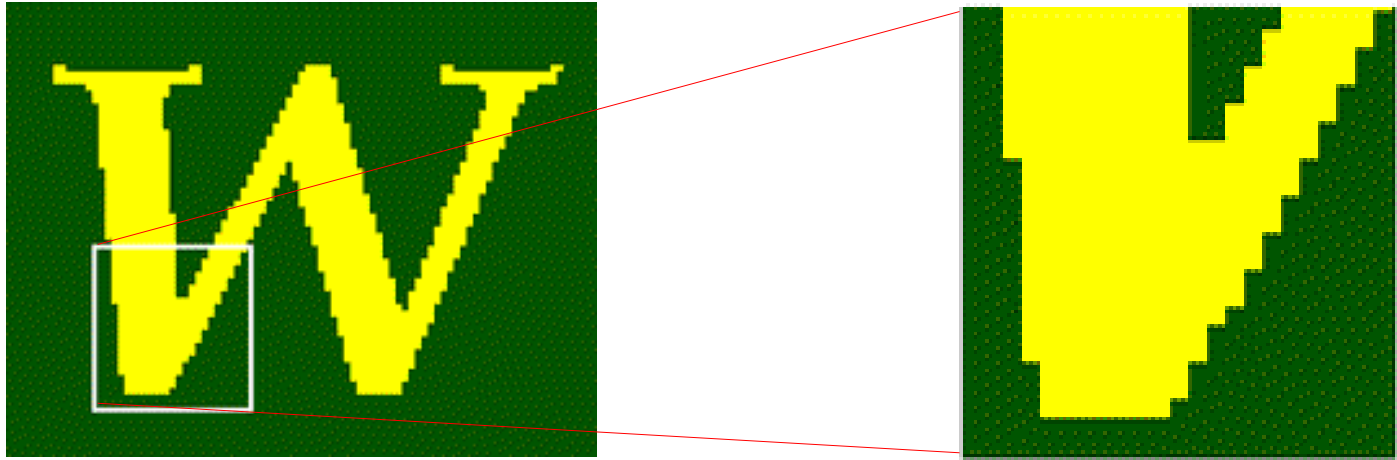
- Ponto / Pixel
- Retas
 - Anti-Aliasing

Anti-Aliasing

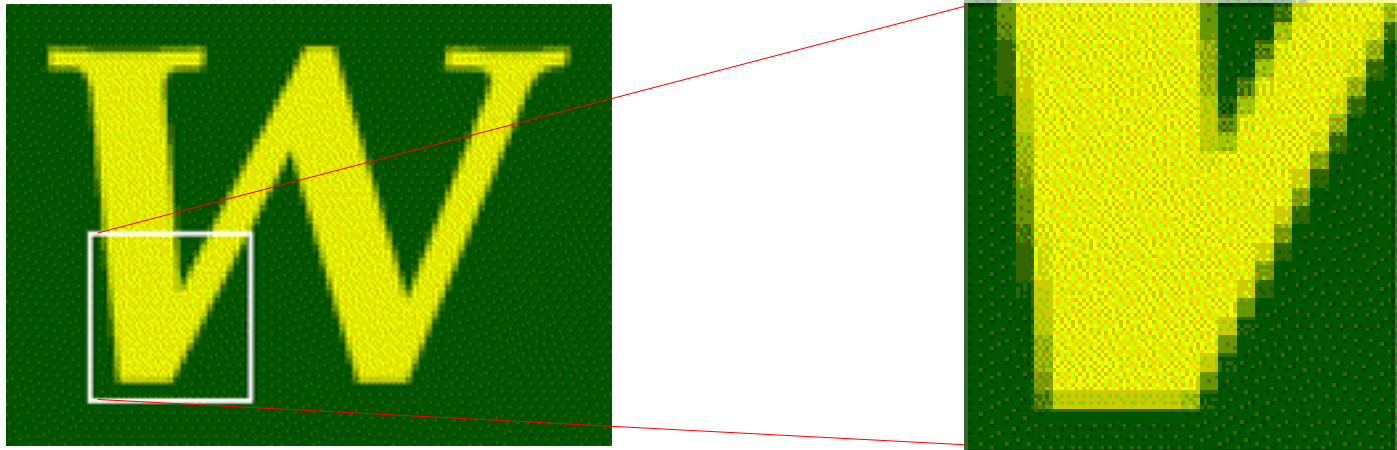
- Quando ampliamos uma imagem digital, notamos um efeito de **serrilhado** na mesma
- Esse é um efeito colateral do **número limitado de pixels**

Anti-Aliasing

Sem
Anti-aliasing

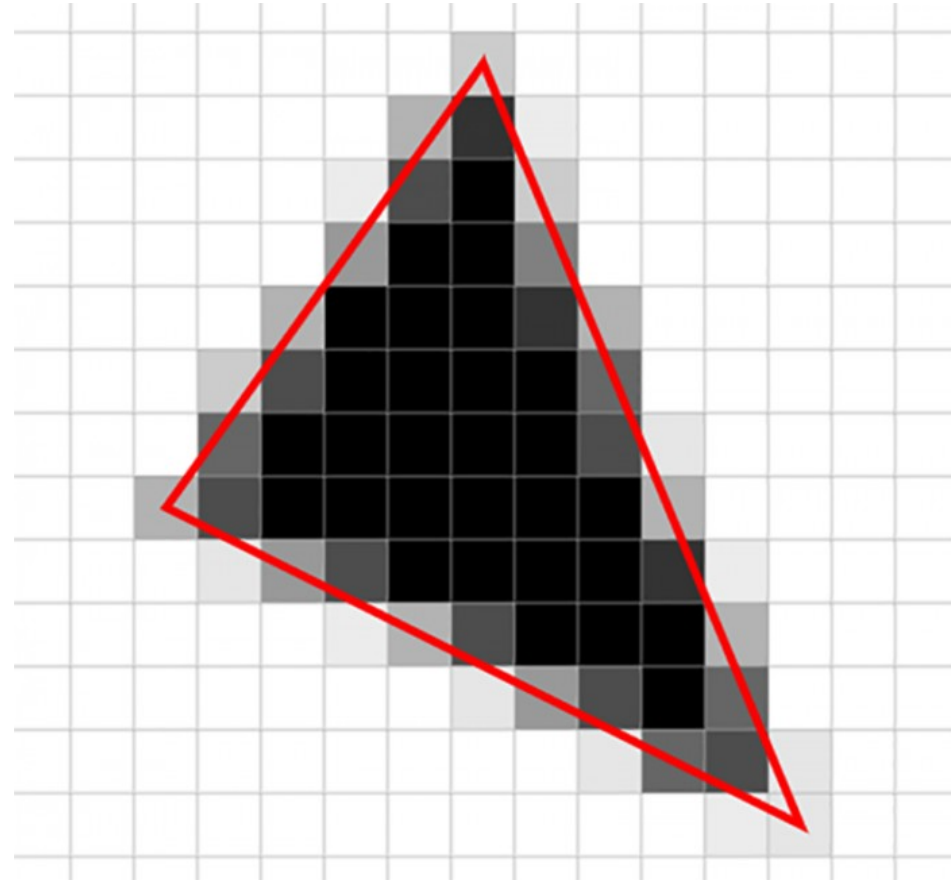


Com
Anti-aliasing



Anti-Aliasing

- Uma das técnicas utilizadas para diminuir esse efeito de serra é “**borrar**” as transições entre pixels de cores diferentes



Anti-Aliasing

- Há diversos algoritmos para realizar o anti-aliasing:
 - Fast approximate anti-aliasing
 - Morphological antialiasing
 - Multisample anti-aliasing
 - Spatial anti-aliasing
 - Supersampling
 - Temporal anti-aliasing
 - Xiaolin Wu's line algorithm
- Quanto maior seu efeito, mais suave a imagem

Anti-Aliasing

- O uso de anti-aliasing requer muito mais processamento, pois calcula **cores médias** entre as cores contrastantes dos pixels, de forma a tornar a transição mais suave
- Isso requer operações de ponto-flutuante.
- Um dos métodos mais utilizados é o algoritmo de **Xiaolin Wu**

Algoritmo de Xiaolin Wu

- **Xiaolin Wu** formou-se em Ciência da Computação na Universidade de Wuhan, em 1982 . Atualmente é PhD e professor na Universidade McMaster, no Canadá
- Em **1991**, criou o algoritmo de anti-aliasing, como melhoria ao algoritmo de **Bresenham**



Algoritmo de Xiaolin Wu

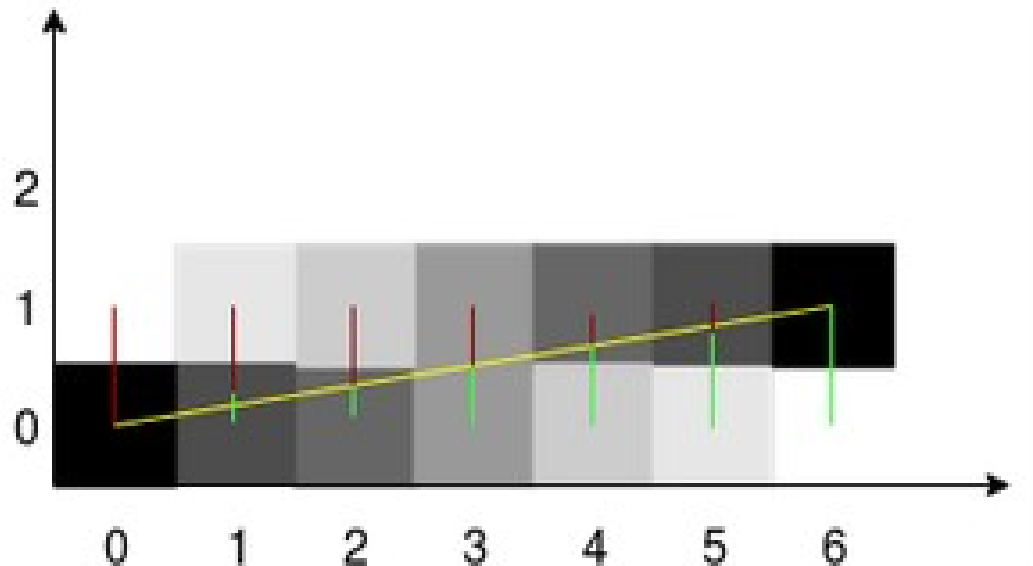
- O algoritmo consiste em **desenhar pares de pixels** abrangendo a linha, cada um colorido de acordo com sua distância da linha.
 - Quanto mais próximo, mais sua cor se aproxima da cor da linha
- Pixels nas extremidades da linha são tratados separadamente.
- Linhas com menos de um pixel são tratadas como um caso especial.

Algoritmo de Xiaolin Wu

- O algoritmo consiste em **desenhar pares de pixels** abrangendo a linha, cada um colorido de acordo com sua distância da linha.
 - Quanto mais próximo, mais sua cor se aproxima da cor da linha
- Pixels nas extremidades da linha são tratados separadamente.
- Linhas com menos de um pixel são tratadas como um caso especial.

Algoritmo de Xiaolin Wu

- Cada etapa do cálculo é realizada para os **dois pixels** mais próximos da linha de pixels, e eles são coloridos com **intensidades diferentes, dependendo da distância.**
- A intensidade do pixel do meio da interseção atual dá 100% se o pixel estiver dentro de 0,9 pixel, a intensidade será de 10%.
- **Em outras palavras, cem por cento da intensidade é dividido entre os pixels que limitam a linha do vetor em ambos os lados.**



Algoritmo de Xiaolin Wu

(1/5)

```
void drawWuLine(int x0 , int y0 , int x1 , int y1)
{
    int steep = absolute(y1 - y0) > absolute(x1 - x0) ;

    // inverte-se as coordenadas | se a inclinação > 1
    // ou se o desenho é da direita para a esquerda
    if (steep)
    {
        swap(&x0 , &y0);
        swap(&x1 , &y1);
    }
    if (x0 > x1)
    {
        swap(&x0 , &x1);
        swap(&y0 , &y1);
    }
}
```

Algoritmo de Xiaolin Wu

(2/5)

```
//calcula-se a inclinação da reta (slope)
float dx = x1-x0;
float dy = y1-y0;
float gradient = dy/dx;
if (dx == 0.0)
    gradient = 1;

int xpxl1 = x0;
int xpxl2 = x1;
float intersectY = y0;
```

Algoritmo de Xiaolin Wu (3/5)

```
// laço principal
if (steep)
{
    int x;
    for (x = xpxl1 ; x <=xpxl2 ; x++)
    {
        // a cobertura do pixel é determinada pela parte
        // decimal da coordenada
        drawPixel(iPartOfNumber(intersectY), x,
                  rfPartOfNumber(intersectY));
        drawPixel(iPartOfNumber(intersectY)-1, x,
                  fPartOfNumber(intersectY));
        intersectY += gradient;
    }
}
```

Algoritmo de Xiaolin Wu

(4/5)

```
else
```

```
{
```

```
    int x;
```

```
    for (x = xpxl1 ; x <=xpxl2 ; x++)
```

```
    {
```

```
        // a cobertura do pixel é determinada pela parte
```

```
        // decimal da coordenada
```

```
        drawPixel(x, iPartOfNumber(intersectY),  
                  rfPartOfNumber(intersectY));
```

```
        drawPixel(x, iPartOfNumber(intersectY)-1,  
                  fPartOfNumber(intersectY));
```

```
        intersectY += gradient;
```

```
    }
```

```
}
```

```
}
```

Algoritmo de Xiaolin Wu

(5/5 : Funções auxiliares)

```
// troca dois números
void swap(int* a , int*b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// arredonda um número
int roundNumber(float x)
{
    return iPartOfNumber(x + 0.5) ;
}
```

```
// retorna a parte decimal de um número
float fPartOfNumber(float x)
{
    if (x>0) return x - iPartOfNumber(x);
    else return x - (iPartOfNumber(x)+1);
}
```

```
// retorna o valor absoluto de um número
float absolute(float x )
{
    if (x < 0) return -x;
    else return x;
}
```

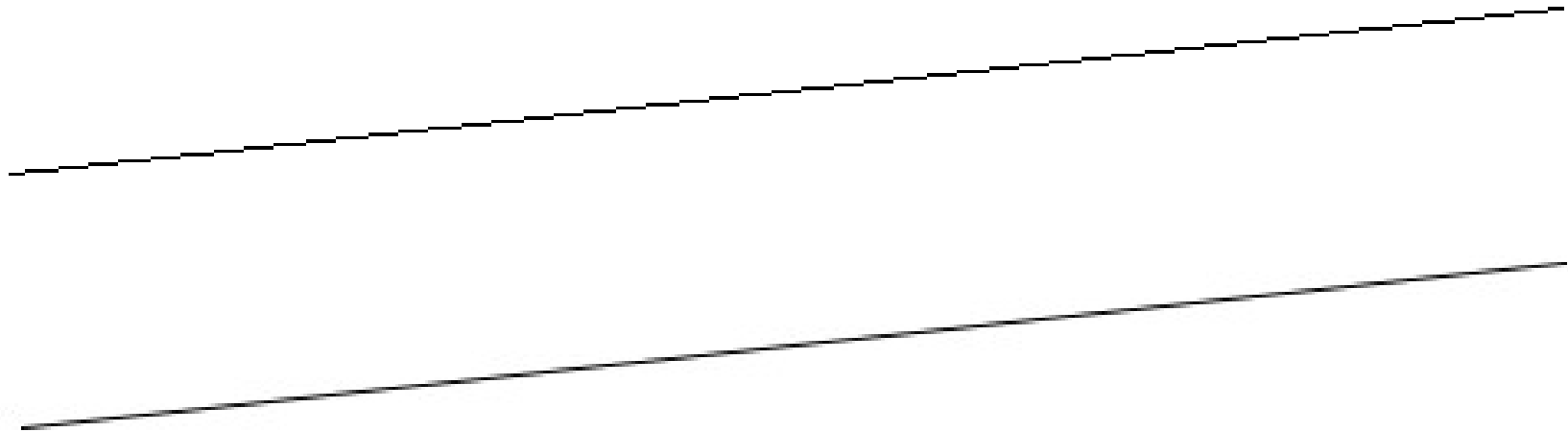
```
// retorna a parte inteira de um número
// de ponto flutuante
int iPartOfNumber(float x)
{
    return (int)x;
}
```

```
// returns 1 - a parte decimal de um número
float rfPartOfNumber(float x)
{
    return 1 - fPartOfNumber(x);
}
```

```
// desenha um pixel na tela com o bilho informado
// 0<=brightness<=1. Usa-a a biblioteca corrente
// para desenhar um ponto na tela
void drawPixel( int x , int y , float brightness)
{
    setPixel(x,y,brightness);
}
```

Efeito e comparação

```
bresenham(80 , 200 , 550, 150);
```



```
drawWuLine(80 , 200 , 550, 150);
```

Dúvidas?



Atividade 1 :

- Siga o roteiro de instalação da ferramenta **CodeBlocks** e da **biblioteca Gráfica SDL**, no documento **Setup CodeBlocks.pdf**

Atividade 2 :

- Crie um projeto chamado **SDL_Points**, e copie para seu arquivo *main.cpp* o conteúdo do arquivo *SDL_points.txt*
- Compile e execute o programa. Verifique os resultados

Atividade 3 :

- Tomando como base o projeto anterior, crie um novo projeto chamado **SDL_Random_Points**, e faça um programa que desenhe pontos em posições e de cores aleatórias na janela.

Atividade 4 :

- Tomando como base o projeto SDL_Points, crie um novo projeto chamado **SDL_Lines**, e implemente nele os algoritmos de ***Bresenham e Xiaolin Wu***

Atividade 5 :

- No projeto **SDL_Lines**, implemente uma função capaz de desenhar retângulos, que recebe por parâmetros dois pontos distintos:
 - o canto superior esquerdo e
 - o canto inferior direito
- Desenhe retângulos diversos

Atividade 6 :

- Ainda no projeto **SDL_Lines**, implemente uma função capaz de desenhar retângulos, preenchidos por uma cor
 - Como fazer? Pintando cada linha em seu interior
- Com as funções que você já possui, faça um desenho clássico de criança, com casinha, cerca e árvore!

